+34 691 225 633
Rev. Febrero 2024

Autor: Jefferson David Rodríguez Chivatá

Dashboard con Plotly

Introducción

Bienvenidos a la segunda guía de profundización de la semana 3, aquí analizaremos con más detalle la herramienta Plotly y Dash que ya definimos en entregas anteriores, y crearemos un pequeño Dashboard a lo largo de este documento. Pondremos en práctica la creación de gráficos con Plotly, el uso de buenas prácticas y el despliegue del dashboard utilizando Dash. Crearemos las gráficas de manera interactiva gracias a las funcionalidades de plotly y también crearemos filtros que nos permitan dividir los datos y obtener gráficos más explicativos de acuerdo a la naturaleza de los datos que usaremos. El dataset que se usará será el "*Icentia11k Single Lead Continuous Raw Electrocardiogram Dataset*", que presenta dos mil millones de latidos pertenecientes a once mil pacientes (No usaremos todos los datos, sólo secciones de estos).

Contenido

Iniciaremos directamente con la creación del código para el dashboard, y se explicará cada parte técnica de él. Adicionalmente se mostrarán los gráficos resultantes de cada parte del código.

1. **Lo primero será importar todas las librerías necesarias para nuestro desarrollo.**

```
# numpy para el manejo de números (cálculos, creación de rangos,
etc)
import numpy as np
# pandas para crear dataframes con nuestro dataset
import pandas as pd
# plotly para crear nuestros gráficos interactivos
import plotly.express as px
# wfdb nos libera de la necesidad de descargar los 188 GB de datos
que posee el dataset original
import wfdb
# Las siguientes herramientas que nos ayudaran con la creación y
estructura de nuestro dashboard final
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
from dash import Dash, html, dcc, callback_context, exceptions
# Estas funcionalidades nos ayudaran con los filtros interactivos
que agregaremos
from dash.dependencies import Input, Output
```

2. **Lo siguiente es obtener los datos con los que trabajaremos**. Para esta tarea no vamos a descargar el dataset entero ya que tiene un tamaño de más de 188 GB, sino que haremos uso de la librería `wfdb` que nos permitirá descargar pequeñas partes del dataset en demanda.

El Dataset que se usará será el *Icentia11k* que contiene las señales continuas de la lectura del electrocardiograma (ECG) de 11.000 pacientes, con un total de 2000 millones de latidos etiquetados. Las señales fueron tomadas con una resolución de 16-bit a 250 Hz, es decir que tendremos 250 datos por segundo.

Los datos están divididos por paciente, y a su vez los datos de cada paciente se dividieron en segmentos de $2^{20} + 1$ muestras que equivalen aproximadamente a 70 minutos cada una. Para esto, se escogieron de manera aleatoria solo 50 segmentos por paciente, y de esta manera reducir la cantidad de datos y mantener una representación acertada de cada paciente.

```
def load_ecg(record_id, segment_id, sampfrom, sampto):

      filename = 'p{:05d}_s{:02d}'.format(record_id, segment_id)
      pn_dir_root = 'icentia11k-continuous-ecg/1.0/'
      pn_dir = 'p{:02d}/p{:05d}/'.format(record_id//1000,record_id)

      signals, fields = wfdb.rdsamp(filename,
                                    pn_dir=pn_dir_root+pn_dir,
                                    sampfrom=sampfrom,
                                    sampto=sampto
                                    )

      df_ecg = pd.DataFrame({'sample': np.arange(sampfrom, sampto),
                             'signal': signals[:,0]})

      return df_ecg
```

Esta función nos retorna un dataframe con datos filtrados por:

- **record_id:** id del paciente
- **segment_id:** número del segmento que queramos usar

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **sampfrom:** minuto inicial que queramos usar (límite inferior del rango)
- **sampto:** minuto final que queramos usar (límite superior del rango)

Para usar esta función podemos ejecutar el siguiente código:

```
df_ecg = load_ecg(18, 0, 2*60*250, 3*60*250)
df_ecg.head()
```

Los minutos que escojamos como parámetros, debemos multiplicarlos por 60 y por 250 que es la frecuencia del medidor de señales:

Figura 1: Estructura de datos de las señales.

	sample	signal
0	30000	-0.026894
1	30001	-0.040342
2	30002	-0.040342
3	30003	-0.040342
4	30004	-0.026894

Fuente: Elaboración propia

El mismo proceso para los latidos etiquetados. Estos datos contienen el momento en el que ocurrió y el tipo de latido (para este ejercicio se restringen los datos sólo a latidos normales (N) y a latidos complejos ventriculares prematuros (V):

```
def load_beats(record_id, segment_id, sampfrom, sampto):

      filename = 'p{:05d}_s{:02d}'.format(record_id, segment_id)
      pn_dir_root = 'icentia11k-continuous-ecg/1.0/'
      pn_dir = 'p{:02d}/p{:05d}/'.format(record_id//1000,record_id)

      ann = wfdb.rdann(filename, "atr",
                  pn_dir=pn_dir_root+pn_dir,
                  sampfrom=sampfrom,
                  sampto=sampto
                  )

      df_beats = pd.DataFrame({'sample': ann.sample,
                               'type': ann.symbol}
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
                                )

        return df_beats

df_beats = load_beats(18, 0, 2*60*250, 3*60*250)
df_beats.head()
```

Figura 2: Estructura de datos de las etiquetas.

	sample	type
0	30069	N
1	30176	V
2	30439	N
3	30620	N
4	30789	N

Fuente: Elaboración propia

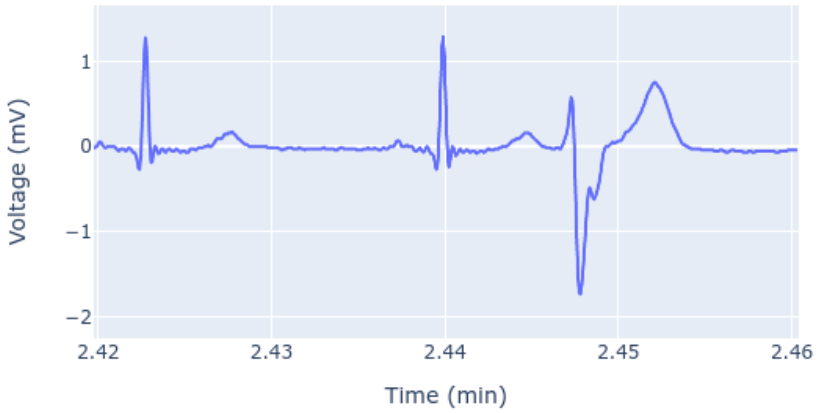
3. **Creación de gráficos a partir de los datos importados:** Con las dos funciones anteriores podemos importar los datos necesarios para crear un par de gráficas interesantes y muy interactivas que se ensamblarán en el Dashboard final. Para esto crearemos un par de funciones extra.

```
def make_ecg_plot(df_ecg):

        # Crea una columna de tiempo en minutos
        df_ecg['Time (min)'] = df_ecg['sample']/250/60

        fig = px.line(df_ecg,
                      x='Time (min)',
                      y='signal',
                      labels={'signal':'Voltage (mV)'},
                      height=350)

        return fig

def make_beat_interval_plot(df_beats):

        # Crea una columna de tiempo en minutos
        df_beats['Time (min)'] = df_beats['sample']/250/60
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
    # Crea una columna para el intervalo de tiempo entre latidos
    df_beats['Interval (s)'] = (df_beats['sample']
                                - df_beats['sample'].shift(1))

    # Crea una columna para el tipo de intervalo
    df_beats['Interval Type'] = (df_beats['type'].shift(1)
                                 + df_beats['type'])

    # Solo considera intervalos entr N y V (NN, NV, VN, VV)
    df_beats = df_beats[
    df_beats['Interval Type'].isin(['NN','NV','VN','VV'])]
    df_beats = df_beats.dropna()

    # Asigna un color a cada tipo de intervalo
    cols = px.colors.qualitative.Plotly
    color_discrete_map = dict(zip(['NN','NV','VN','VV'],
cols[:4]))
    fig = px.scatter(df_beats,
                     x='Time (min)',
                     y='Interval (s)',
                     color='Interval Type',
                     color_discrete_map=color_discrete_map,
                     height=350
                     )

    return fig
```

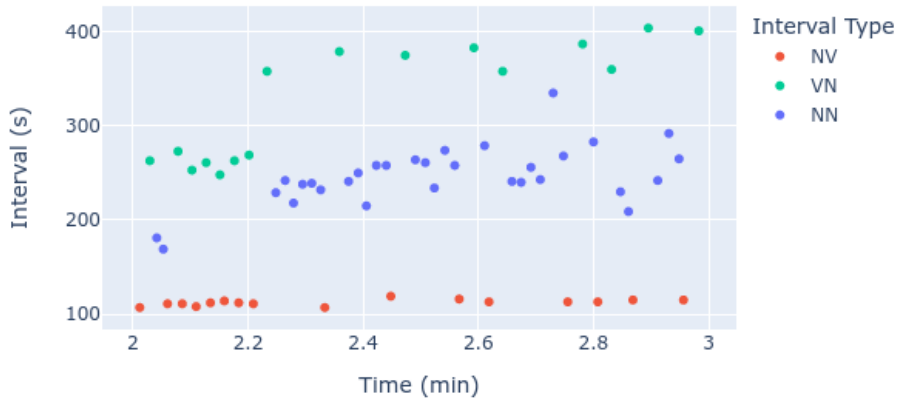
Figura 2: Visualización interactiva de la señal de ECG con Plotly.



Fuente: Elaboración propia

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 3: Visualización interactiva de los intervalos de los latidos con Plotly.



Fuente: Elaboración propia

4. Creación del Dashboard

Hasta ahora hemos:

- Importado las librerías necesarias
- Creado un par de funciones para importar los datos necesarios
- Creado un par de funciones para crear los gráficos que mostraremos

Nos falta ensamblar todo en un mismo tablero y crear los filtros interactivos. Todo debe quedar en un solo script de python o en su defecto en una única celda en el notebook.

```
import numpy as np
import pandas as pd
import plotly.express as px
import wfdb

from dash import Dash, html, dcc, callback_context, exceptions
from dash.dependencies import Input, Output

css = 'https://codepen.io/chriddyp/pen/bWLwgP.css'
app = Dash(__name__, external_stylesheets=[css])
server = app.server

#####################################
# Ponemos nuestras 4 funciones aquí #
#####################################
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
################################
app.layout = html.Div([
      # List all components of app here
      html.H3('ECG Dashboard'),
      # Agrega estilo al dashboard
      style={'width':'200px',
             'height':'60px',
             'padding-left':'2%',
             'display':'inline-block',}

      # Dentro de este div ponemos todos los elementos que se
      # verán dentro de nuestro dashboard. Cada gráfico, filtro,
      # titulo, etc debe estar dentro de un Div
])
################################

if __name__ == '__main__':
      app.run_server(debug=True)
```

Para visualizar el dashboard que estamos creando es necesario guardar el código en un archivo llamado `app.py` y se debe lanzar con el comando (en cualquier momento) `python app.py` o ejecutar la celda del notebook.

Para agregar nuestras gráficas debemos cargar nuestros datos y crear las figuras:

```
# Definimos los valores/parámetros por defecto
record_id_def = 0
segment_id_def = 0

# Cargamos los datos
df_beats = load_beats(record_id_def, segment_id_def, 0, None)
df_ecg = pd.DataFrame({'sample':[], 'signal':[]})

# Creamos las figuras que se mostrarán por defecto al inicializar
# el dashboard
fig_intervals = make_beat_interval_plot(df_beats)
fig_ecg = make_ecg_plot(df_ecg)
```

Y Para agregarlos al dashboard basta con agregar las siguientes instrucciones al `app.layout`:

```
html.Div(
      dcc.Graph(id='fig_intervals', figure=fig_intervals),
),
html.Div(
      dcc.Graph(id='fig_ecg', figure=fig_ecg),
)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **Ahora agregaremos los filtros con sus respectivos *callbacks***

Los *callbacks* no son más que funciones que nos permitirán tomar los valores de los filtros y hacer que nuestros gráficos se actualicen de acuerdo al valor actual de los filtros. Se agregarán al `layout` dos filtros de tipo *dropdown* a nuestro dashboard:

```
html.Div([
      html.Label('Record ID'),
      dcc.Dropdown(id='dropdown_record_id',
                   value=record_id_def,
                   options=np.arange(11000))
]),
html.Div([
      html.Label('Segment ID'),
      dcc.Dropdown(id='dropdown_segment_id',
                   options=np.arange(50),
                   value=segment_id_def)
]),
```

Los *callbacks* se agregan como decoradores de python como se evidencia en el siguiente código:

```
@app.callback(
      Output('fig_intervals','figure'),
      Input('dropdown_record_id','value'),
      Input('dropdown_segment_id','value')
)
def update_record(record_id, segment_id):

      # Si la caja del menú se limpio, no haga nada
      if (record_id is None) or (segment_id is None):
      raise exceptions.PreventUpdate()

      df_beats = load_beats(record_id, segment_id, 0, None)
      fig_intervals = make_beat_interval_plot(df_beats)

      return fig_intervals

def update_ecg_plot(relayout_data, record_id, segment_id):
    # callback_context proporciona información sobre la entrada que
se ha activado
    ctx = callback_context
    # Si se ha activado layout_data
    if ctx.triggered[0]['prop_id'] == 'fig_intervals.relayoutData':
        if relayout_data==None:
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
            relayout_data={}
        # # Si no ha cambiado ninguno de los límites (puede que se
haya pulsado otro botón)
        # entonces impedir la actualización
        if ('xaxis.range[0]' not in relayout_data) and \
           ('xaxis.range[1]' not in relayout_data) and \
           ('xaxis.autorange' not in relayout_data):
            raise exceptions.PreventUpdate()
        # Si se ha pulsado el botón de alcance automático
        if 'xaxis.autorange' in relayout_data:
            tmin_adjust = 0
            tmax_adjust = 60
        # Si se ha actualizado el límite inferior
        if ('xaxis.range[0]' in relayout_data):
            tmin_adjust = relayout_data['xaxis.range[0]']
        else:
            tmin_adjust = 0
        # Si se ha actualizado el límite superior
        if ('xaxis.range[1]' in relayout_data):
            tmax_adjust = relayout_data['xaxis.range[1]']
        else:
            tmax_adjust = 60
    # Si se activó record_id o segment_id, reiniciar ECG
    else:
        tmin_adjust = 0
        tmax_adjust = 60
    # Si la ventana temporal es < 1 min, importar datos de ECG
    if tmax_adjust - tmin_adjust < 1:
        sampfrom = int(tmin_adjust*60*250)
        sampto = int(tmax_adjust*60*250)
        df_ecg = load_ecg(record_id, segment_id, sampfrom, sampto)
        # Elaborar la figura ECG
        fig_ecg = make_ecg_plot(df_ecg)
    else:
        # Crear la figura de ECG vacía
        df_ecg = pd.DataFrame({'sample':[], 'signal':[]})
        fig_ecg = make_ecg_plot(df_ecg)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

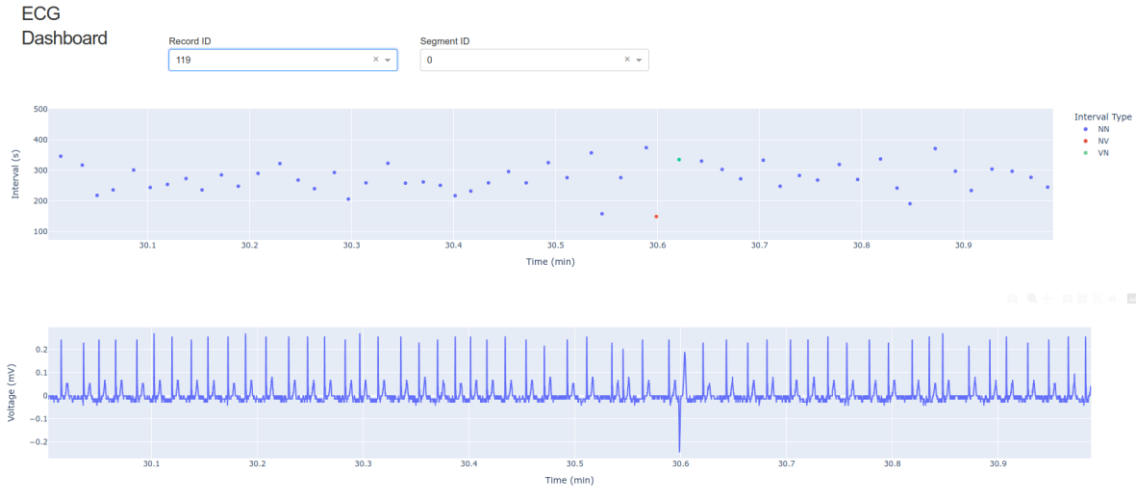
```
    return fig_ecg
```

Los Input serán los id de los filtros *dropdown,* y el output será la figura que se mostrará en el dashboard. El segundo *callback* agregado reaccionará al cambio en el zoom de la primera gráfica, es decir, cuando en la primera gráfica se seleccione una porción de máximo 1 minuto, la segunda gráfica empezará a mostrarse. De otro modo siempre estará vacía.

Nota 1: Las funciones de *callback* se deben agregar después del elemento layout.

El resultado final al ejecutar el código se debería ver así:

Figura 4: Dashboard final con Plotly y Dash



Fuente: Elaboración propia.

Conclusiones

- Al terminar este documento se exploró la creación de un dashboard interactivo con Plotly y Dash de principio a fin.
- La creación de dashboard es sencilla siempre y cuando los datos que se usarán estén organizados y listos.
- Plotly y Dash nos brindan funcionalidades muy poderosas para crear dashboards estéticos e interactivos en poco tiempo y con pocas líneas de código.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Referencias

PhysioNet, MIT Laboratory for Computational Physiology. Recuperado el 20 de Enero de 2024 de https://physionet.org/content/icentia11k-continuous-ecg/1.0/p00/p00000/#files-panel

Medium, Building a Dashboard in Plotly Dash, Thomas Bury, Recuperado el 20 de Enero de 2024 de https://towardsdatascience.com/building-a-dashboard-in-plotly-dash-c748588e2920

Plotly, Inc. Advance callbacks, Recuperado el 20 de Enero de 2024 de https://dash.plotly.com/advanced-callbacks

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.